

DATABASE MANAGEMENT SYSTEMS

Complete Technical Course Handbook & Detailed Syllabus Notes

Course Code: CS-310

Teaching Scheme: 3-1-0 (L-T-P)

Coverage: Units I through V (End-to-End)

UNIT I: Introduction & Entity Relationship Data Modeling

1. Overview & File System vs. Database Management System

A **Database Management System (DBMS)** is a comprehensive software system that enables users to define, create, maintain, and control structured access to a centralized or distributed database. Prior to DBMS, organizations relied on traditional file-processing systems where records were stored in plain operating system files.

Characteristic	Traditional File System	Database Management System (DBMS)
Data Redundancy & Inconsistency	High redundancy as multiple applications maintain separate copies of overlapping data, leading to inconsistency.	Centralized storage minimizes redundancy; automated constraints ensure data consistency across views.
Data Isolation & Access	Data scattered across different files and formats; writing new application programs is required for ad-hoc queries.	Provides unified query languages (e.g., SQL) enabling efficient, declarative ad-hoc data retrieval.
Concurrent Access	Difficult to coordinate concurrent updates; risk of race conditions, lost updates, and corrupted data files.	Advanced concurrency control mechanisms (e.g., Two-Phase Locking, MVCC) ensure safe concurrent execution.
Atomicity & Recovery	Partial file updates during system failures leave data in an inconsistent state without automatic rollback.	Guarantees atomic transactions via write-ahead logging (WAL) and automated recovery systems.
Data Integrity	Integrity constraints are hardcoded into individual application programs, making modification complex.	Constraints (PRIMARY KEY, FOREIGN KEY, CHECK) are defined centrally in the schema and enforced by DBMS.
Security & Authorization	File-level access controls provided by OS; coarse-grained security with limited user-level restriction.	Granular access controls (GRANT/REVOKE) at column, row, table, and view levels.

2. Database System Architecture & Data Independence

The ANSI/SPARC **Three-Schema Architecture** abstracts physical data storage details from end users:

- **Internal / Physical Level:** Describes the physical storage structure, access paths, indexing structures (B+ trees), data compression, and allocation of disk blocks.
- **Conceptual / Logical Level:** Describes *what* data is stored in the database and relationships among data. Represents tables, attributes, constraints, and relationships without physical details.
- **External / View Level:** Highest level of abstraction. Defines customized views for different user groups, shielding unused fields and sensitive data.

Data Independence

Data independence refers to the capacity to modify a schema at one level without forcing alterations in the schema at the next higher level:

- **Physical Data Independence:** The ability to modify the physical schema (e.g., reindexing, altering storage paths, moving to new storage media) without changing the logical/conceptual schema or application code.
- **Logical Data Independence:** The ability to modify the conceptual schema (e.g., adding/deleting entities, attributes, or relationships) without requiring changes to existing external views or user programs.

3. Database Languages & Overall Structure

- **Data Definition Language (DDL):** Used to specify schema structures (`CREATE` , `ALTER` , `DROP` , `TRUNCATE`). DDL output is compiled into the Data Dictionary (Metadata).
- **Data Manipulation Language (DML):** Used for accessing and manipulating data (`INSERT` , `UPDATE` , `DELETE` , `SELECT`). Can be *procedural* (user specifies what and how) or *declarative/non-procedural* (user specifies what data is needed, DBMS optimizes access).
- **Data Control Language (DCL) & TCL:** DCL manages authorization (`GRANT` , `REVOKE`); TCL manages transactions (`COMMIT` , `ROLLBACK` , `SAVEPOINT`).

Overall DBMS Structure Component Breakdown:

1. **Query Processor:** Contains DDL Interpreter, DML Compiler (translates DML into low-level evaluation plans), Query Optimizer (evaluates query plans based on cost), and Query Evaluation Engine.
2. **Storage Manager:** Interfaces between low-level data and application programs. Modules include Authorization and Integrity Manager, Transaction Manager, Buffer Manager, and File Manager.

4. Data Modeling Using the Entity-Relationship (ER) Model

The ER model models real-world enterprise structures using entities, attributes, and relationships.

- **Entity & Entity Set:** An entity is an object identifiable from other objects. An entity set is a collection of similar entities (e.g., `STUDENT` , `COURSE`).
- **Attributes:**
 - *Simple / Atomic:* Cannot be divided (e.g., `Age`).
 - *Composite:* Subdivided into smaller components (e.g., `Name` → `First_Name` , `Last_Name`).
 - *Single-Valued vs. Multi-Valued:* Multi-valued can store multiple values (e.g., `{Phone_Number}` , represented by double ellipses).
 - *Derived Attribute:* Computed from other attributes (e.g., `Age` derived from `DOB` , dashed ellipse).
 - *Key Attribute:* Uniquely identifies an entity instance (underlined text).
- **Weak Entity Sets:** An entity set that lacks a primary key. It relies on an *Identifying Owner Entity Set* through an *Identifying Relationship* (double diamond). Its discriminator (partial key) is indicated with a dashed underline.

5. Mapping Constraints, Keys & ER Notation

- **Mapping Cardinality Ratios:** Expresses the number of entities to which another entity can be associated via a relationship set: **One-to-One (1:1)**, **One-to-Many (1:N)**, **Many-to-One (N:1)**, and **Many-to-Many (M:N)**.
- **Participation Constraints:**
 - *Total Participation (Existence Dependency):* Every entity in the set must participate in at least one relationship (represented by double lines).
 - *Partial Participation:* Some entities may not participate in the relationship set.
- **Key Definitions:**
 - **Super Key (SK):** A set of one or more attributes that uniquely identifies a record in a relation.
 - **Candidate Key (CK):** A minimal Super Key (no proper subset is a Super Key).

- **Primary Key (PK):** A candidate key selected by the database designer as the principal tuple identifier.
- **Foreign Key (FK):** An attribute in a table that references the Primary Key of another table.

6. Extended ER (EER) Concepts & Reduction of ER Diagrams to Relational Tables

- **Specialization:** Top-down process of creating lower-level sub-entities from a higher-level entity based on distinguishing characteristics.
- **Generalization:** Bottom-up synthesis combining multiple lower-level entity sets with common features into a higher-level super-entity set.
- **Aggregation:** An abstraction through which relationships are treated as higher-level entities, enabling them to participate in secondary relationships.
- **Higher-Degree Relationships:** Relationships involving 3 (Ternary) or more entity sets (N-ary).

Algorithm: Reduction of ER Diagram to Relational Schema

1. **Strong Entity Set:** Converts directly into a table with scalar attributes. Composite attributes are flattened into component columns. Multi-valued attributes are separated into a new table with the entity's Primary Key.
2. **Weak Entity Set:** Becomes a table containing all its scalar attributes plus the Primary Key of the identifying strong owner entity set as a Foreign Key. The Primary Key is (Owner PK + Discriminator).
3. **1:N Relationship Set:** Place the Primary Key of the "1" side as a Foreign Key in the table on the "N" side along with any relationship attributes.
4. **M:N Relationship Set:** Create a separate relationship table whose primary key is the combination of Primary Keys of both participating entity sets.
5. **1:1 Relationship Set:** Merge tables if participation is total on both sides, or place the Foreign Key on the total-participation side.

UNIT II: Relational Model, Relational Algebra, Calculus & SQL/PL-SQL

1. Relational Data Model Concepts & Integrity Constraints

In the relational model, data is organized into two-dimensional tables called **Relations**. A row is a **Tuple** and a column is an **Attribute**.

- **Domain Constraints:** Every column value must belong to an atomic domain of permissible values (e.g., data types, range bounds).
- **Key Constraints:** Ensures no two distinct tuples in a relation instance have identical values for candidate key attributes.
- **Entity Integrity Constraint:** Primary key attributes cannot contain **NULL** values, as NULL prevents unique tuple identification.
- **Referential Integrity Constraint:** A foreign key value in a referencing relation must either match an existing primary key value in the referenced relation or be completely **NULL**.

2. Relational Algebra (Procedural Query Language)

Relational algebra operates on relations and returns relations. Operations are categorized into Fundamental and Derived operators:

Operator	Symbol & Syntax	Description & Formal Rule
Selection	$\sigma_p(R)$	Filters tuples from relation R that satisfy predicate p .
Projection	$\pi_{A1, A2}(R)$	Selects specified columns A1, A2 and automatically eliminates duplicate rows.

Operator	Symbol & Syntax	Description & Formal Rule
Union	$R \cup S$	Combines tuples from R and S . Requires R and S to be union-compatible (same arity and compatible domains).
Set Difference	$R - S$	Returns tuples present in R but not in S . Requires union compatibility.
Cartesian Product	$R \times S$	Pairs every tuple in R with every tuple in S . Result schema has degree $deg(R) + deg(S)$.
Rename	$\rho_x(R)$	Renames relation R to new designation x .
Natural Join	$R \bowtie S$	Combines matching tuples from R and S over common attribute names, removing duplicate join columns.
Theta Join	$R \bowtie_{\theta} S$	Combines tuples satisfying condition θ : $\sigma_{\theta}(R \times S)$.
Division	$R \div S$	Finds tuples in R that pair with <i>all</i> tuples in S . Useful for queries involving "ALL".

3. Relational Calculus (Non-Procedural Query Language)

Relational calculus specifies *what* information is desired without detailing the procedure to retrieve it.

- **Tuple Relational Calculus (TRC):** Expressions are of the form $\{t \mid P(t)\}$, representing the set of all tuples t such that predicate condition $P(t)$ is true. Uses quantifiers: Existential ($\exists$) and Universal ($\forall$).
Example: Find students in 'CS' department: $\{t \mid t \in STUDENT \wedge t.Dept = 'CS'\}$
- **Domain Relational Calculus (DRC):** Expressions use domain variables: $\{ \{x_1, x_2, \dots, x_n\} \mid P(x_1, x_2, \dots, x_n) \}$.
- **Safety of Expressions:** A calculus expression is safe if it produces a finite result. Unsafe queries (e.g., $\{t \mid \neg(t \in STUDENT)\}$) return infinite results and must be restricted by language bounds.

4. Structured Query Language (SQL) & Database Objects

SQL is the standard language for relational database management systems.

Categorization of SQL Commands

- **DDL:** CREATE, ALTER, DROP, TRUNCATE, RENAME
- **DML:** INSERT, UPDATE, DELETE
- **DQL:** SELECT (with clauses WHERE, GROUP BY, HAVING, ORDER BY)
- **DCL / TCL:** GRANT, REVOKE / COMMIT, ROLLBACK, SAVEPOINT

```
-- Example Schema Creation with Constraints
CREATE TABLE Department (
    dept_id NUMBER(4) PRIMARY KEY,
    dept_name VARCHAR2(50) NOT NULL UNIQUE
);

CREATE TABLE Employee (
    emp_id NUMBER(6) PRIMARY KEY,
    emp_name VARCHAR2(100) NOT NULL,
    salary NUMBER(10,2) CHECK (salary > 0),
    dept_id NUMBER(4),
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES Department(dept_id) ON DELETE SET NULL
);

-- Advanced Query with Subquery, Joins, and Aggregates
```

```

SELECT d.dept_name, COUNT(e.emp_id) AS total_emp, AVG(e.salary) AS avg_sal
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM Employee)
ORDER BY avg_sal DESC;

```

Views & Indexes

- **Views:** Virtual tables created via `CREATE VIEW view_name AS SELECT...`. They enhance security, simplify complex queries, and isolate application code from logical schema changes. Updatable views must be defined over a single table without aggregates or distinct clauses.
- **Indexes:** Data structures (typically B-Trees or Hash tables) created via `CREATE INDEX idx_emp_sal ON Employee(salary);` to accelerate query retrieval at the cost of slight write overhead during inserts/updates.

5. PL/SQL Programming (Procedural Extension to SQL)

PL/SQL extends SQL with procedural capabilities (variables, loops, conditionals, exception handling).

```

-- PL/SQL Stored Procedure with Explicit Cursor & Exception Handling
CREATE OR REPLACE PROCEDURE Give_Bonus (p_dept_id IN NUMBER, p_percent IN NUMBER) IS
    CURSOR c_emp IS
        SELECT emp_id, salary FROM Employee WHERE dept_id = p_dept_id FOR UPDATE;
    v_emp_id Employee.emp_id%TYPE;
    v_salary Employee.salary%TYPE;
    e_invalid_percent EXCEPTION;
BEGIN
    IF p_percent <= 0 THEN
        RAISE e_invalid_percent;
    END IF;

    OPEN c_emp;
    LOOP
        FETCH c_emp INTO v_emp_id, v_salary;
        EXIT WHEN c_emp%NOTFOUND;

        UPDATE Employee
        SET salary = salary + (salary * (p_percent / 100))
        WHERE CURRENT OF c_emp;
    END LOOP;
    CLOSE c_emp;
    COMMIT;
EXCEPTION
    WHEN e_invalid_percent THEN
        DBMS_OUTPUT.PUT_LINE('Error: Percentage must be greater than zero.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END Give_Bonus;
/

```

Triggers: Stored programs executed automatically in response to DML events (`BEFORE / AFTER INSERT/UPDATE/DELETE`). Row-level triggers use `:NEW.col` and `:OLD.col` pseudo-records to inspect or enforce complex audit constraints.

UNIT III: Database Design & Normalization Theory

1. Functional Dependencies (FD) & Armstrong's Axioms

A Functional Dependency $X \rightarrow Y$ asserts that if two tuples in a relation R agree on attributes X , they must agree on attributes Y .

Armstrong's Axioms (Sound & Complete Inference Rules)

- **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$.
- **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any attribute set Z .
- **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Secondary Rules: Union (If $X \rightarrow Y$ and $X \rightarrow Z \Rightarrow X \rightarrow YZ$), Decomposition (If $X \rightarrow YZ \Rightarrow X \rightarrow Y$ and $X \rightarrow Z$), Pseudo-transitivity (If $X \rightarrow Y$ and $WY \rightarrow Z \Rightarrow WX \rightarrow Z$).

Attribute Closure Algorithm (X^+)

To compute the attribute closure X^+ under a set of FDs F :

1. Initialize $X^+ = X$.
2. Repeat until no new attributes can be added: For each FD $A \rightarrow B$ in F , if $A \subseteq X^+$, then set $X^+ = X^+ \cup B$.
3. If X^+ contains all attributes of relation R , then X is a **Super Key**.

2. Normalization & Normal Forms (1NF, 2NF, 3NF, BCNF)

Normalization reduces data redundancy and eliminates update anomalies (Insertion, Deletion, Update Anomalies).

Normal Form	Requirement / Condition	Target Anomaly Eliminated
1NF	All attributes must contain atomic (indivisible) values. No repeating groups or multi-valued attributes allowed.	Eliminates nested structures and non-atomic attribute arrays.
2NF	Must be in 1NF. Every non-prime attribute must be <i>fully functionally dependent</i> on the primary key (No partial dependency: $X \rightarrow Y$ where X is a proper subset of a Candidate Key and Y is non-prime).	Eliminates redundant storage of data dependent on partial key components.
3NF	Must be in 2NF. No non-prime attribute should be transitively dependent on any Candidate Key. For every non-trivial FD $X \rightarrow Y$, either X is a Super Key or Y is a prime attribute.	Eliminates transitive dependencies across non-key attributes.
BCNF	Strict variant of 3NF. For <i>every</i> non-trivial FD $X \rightarrow Y$, X MUST be a Super Key (no exceptions for prime attributes).	Eliminates all redundancy arising from functional dependencies.
4NF	Must be in BCNF. Eliminates non-trivial Multi-valued Dependencies (MVD) $X \twoheadrightarrow Y$ where X is not a super key.	Eliminates independent multi-valued facts stored in a single table.

Normal Form	Requirement / Condition	Target Anomaly Eliminated
5NF (PJNF)	Must be in 4NF. Eliminates Join Dependencies (JD) that do not arise from candidate key constraints.	Prevents cyclic join anomalies across 3 or more decomposed relations.

3. Decomposition Properties: Lossless Join & Dependency Preservation

When decomposing a relation **R** into **R1** and **R2**:

- **Lossless-Join Decomposition:** Guarantees that joining the decomposed tables recovers the exact original relation without spurious tuples ($R1 \bowtie R2 = R$).

Theorem: A decomposition of **R** into (**R1**, **R2**) is lossless iff:

$$(R1 \cap R2) \rightarrow R1 \quad \text{OR} \quad (R1 \cap R2) \rightarrow R2$$

- **Dependency Preservation:** Ensures that all functional dependencies in **F** can be tested/enforced locally on individual decomposed relations without executing expensive join operations:

$$(F1 \cup F2)^+ = F^+$$

Important Decomposition Tradeoff

Any relation schema can be decomposed into **3NF** while simultaneously ensuring *both* Lossless Join and Dependency Preservation. However, BCNF guarantees Lossless Join but cannot always guarantee Dependency Preservation.

UNIT IV: Transaction Processing, Recovery & Distributed Databases

1. Transaction Processing & ACID Properties

A **Transaction** is a logical unit of database processing that includes one or more database access operations (Read/Write).

- **Atomicity:** "All or Nothing" principle. Either all operations execute successfully or the entire transaction is rolled back. (Managed by Recovery Manager via logs).
- **Consistency:** Transaction preserves database invariants. Transforms DB from one consistent state to another. (Managed by Application Programmers & Constraint Enforcement).
- **Isolation:** Execution of one transaction is unaware of concurrent transactions. Intermediate states remain invisible. (Managed by Concurrency Control Manager).
- **Durability:** Once committed, changes persist permanently even in the presence of system failures. (Managed by Recovery Manager & Persistent Log File).

Transaction State Transition Diagram

Active → Partially Committed (after final operation executed) → Committed.

If checks fail: Active / Partially Committed → Failed → Aborted (Rollback completed).

2. Schedules & Serializability

A **Schedule** represents the chronological execution sequence of operations from interleaved transactions.

- **Conflict Serializability:** Two operations in a schedule conflict if they belong to different transactions, access the same data item, and at least one operation is a Write (**W(X)**).

A schedule is Conflict Serializable if it is conflict equivalent to a serial schedule.

Testing Conflict Serializability: Precedence Graph Algorithm

1. Create a node for each transaction T_i in the schedule.
2. Draw a directed edge $T_i \rightarrow T_j$ if T_i executes an operation $O1$ before T_j executes $O2$, $O1$ and $O2$ conflict, and both access the same item.
3. **Result:** If the Precedence Graph contains **NO CYCLES**, the schedule is Conflict Serializable. Topological sorting yields equivalent serial execution orders.

- **View Serializability:** Broader class of serializability allowing blind writes. Every conflict serializable schedule is view serializable. Testing view serializability is NP-Complete.
- **Recoverability Classification:**
 - *Recoverable Schedule:* If T_j reads data written by T_i , then T_i must commit BEFORE T_j commits.
 - *Cascadeless Schedule (Avoids Cascading Aborts - ACA):* If T_j reads data written by T_i , T_i must commit BEFORE T_j performs the read.
 - *Strict Schedule:* No transaction can read or write an uncommitted item written by another transaction.

3. Recovery Systems & Deadlock Handling

- **Log-Based Recovery Techniques:** Uses Write-Ahead Logging (WAL) protocol (log record written to persistent disk before actual database modifications occur).
 - *Deferred Update (NO-UNDO/REDO):* Database writes are deferred until commit. Aborts require no UNDO; committed transactions are REDONE from log upon crash.
 - *Immediate Update (UNDO/REDO):* Database modified during execution. Uncommitted transactions require UNDO; committed transactions require REDO.
- **Checkpoints:** Periodically flush memory buffers and log records to disk, writing a `<CHECKPOINT>` log record. Limits search distance during crash recovery.
- **Deadlock Handling:**
 - *Prevention Protocols (Timestamp-based):*
 - Wait-Die (Non-preemptive):** Older transaction waits for younger; younger transaction dies when requesting older's lock.
 - Wound-Wait (Preemptive):** Older transaction wounds (preempts/aborts) younger; younger waits for older.
 - *Detection:* Periodically construct Wait-For Graphs (WFG). Cycles indicate deadlock.
 - *Recovery:* Select a victim, rollback transaction to checkpoint/beginning, avoid starvation.

4. Distributed Database Systems (DDBMS)

- **Distributed Data Storage:**
 - *Horizontal Fragmentation:* Relation split into subsets of rows (tuples) using selection conditions.
 - *Vertical Fragmentation:* Relation split into subsets of columns (attributes) maintaining primary keys.
 - *Replication:* Storing duplicate fragments across multiple geographic sites to enhance availability and read latency.
- **Distributed Concurrency Control:** Requires distributed 2PL, Primary Site Locking, or 2-Phase Commit (2PC) protocol to guarantee distributed transaction atomicity across network partitions.

UNIT V: Concurrency Control Techniques & Oracle Case Study

1. Locking Techniques for Concurrency Control

Locks control concurrent access to database items. Basic lock modes: **Shared (S)** for reading, **Exclusive (X)** for writing.

Requested \ Granted	Shared (S)	Exclusive (X)
Shared (S)	Compatible	Conflict
Exclusive (X)	Conflict	Conflict

Two-Phase Locking (2PL) Protocol

Guarantees Conflict Serializability by forcing transactions to acquire and release locks in two distinct phases:

1. **Growing Phase:** Transaction acquires locks as needed; cannot release any lock. Lock upgrades ($S \rightarrow X$) permitted.
2. **Shrinking Phase:** Transaction releases locks; cannot acquire any new lock. Lock downgrades ($X \rightarrow S$) permitted.

2PL Variations

- **Strict 2PL:** All Exclusive (X) locks acquired by a transaction must be held until the transaction COMMITS or ABORTS. Prevents cascading rollbacks.
- **Rigorous 2PL:** ALL locks (both Shared and Exclusive) are held until transaction commit/abort. Ensures strict serial order matching commit order.

2. Timestamp-Based & Validation Protocols

- **Timestamp Ordering (TO) Protocol:** Each transaction T_i is assigned a unique logical timestamp $TS(T_i)$ upon entering the system.
For data item Q , system maintains $R-TS(Q)$ (largest timestamp of transaction that read Q) and $W-TS(Q)$ (largest timestamp of transaction that wrote Q).
 - Read Rule: If $TS(T_i) < W-TS(Q)$, reject read and abort T_i . Else execute read and set $R-TS(Q) = \max(R-TS(Q), TS(T_i))$.
 - Write Rule: If $TS(T_i) < R-TS(Q)$ or $TS(T_i) < W-TS(Q)$, reject write and abort T_i . Else execute write and update $W-TS(Q)$.
- **Thomas' Write Rule:** Optimization for TO protocol. If $TS(T_i) < W-TS(Q)$ during a Write operation, instead of aborting T_i , the write is simply ignored (obsolete write). Allows non-serializable execution schedules that remain view serializable.
- **Validation-Based (Optimistic) Protocol:** Designed for read-heavy workloads with low conflict likelihood. Execution occurs in 3 phases:
 1. Read Phase: Reads values into local workspace; writes performed on local temp copies.
 2. Validation Phase: Checks if updates conflict with concurrent committed transactions.
 3. Write Phase: If validation succeeds, changes applied to DB; otherwise aborted.

3. Multiple Granularity Locking (MGL) & Multi-Version Schemes

MGL allows data items of various sizes (Database $\rightarrow$ Area $\rightarrow$ File $\rightarrow$ Record) to be locked, represented as a tree structure. To support fine-grained locking efficiently, **Intent Locks** are introduced:

- **Intent Shared (IS):** Explicit locking taking place at lower levels with shared locks.
- **Intent Exclusive (IX):** Explicit locking taking place at lower levels with exclusive locks.
- **Shared Intent Exclusive (SIX):** Explicit shared locking at current node, with explicit exclusive locks at lower levels.

Multi-Version Concurrency Control (MVCC): Maintains multiple historical versions of data items. When a write operation occurs, a new version is created. Read requests are routed to appropriate snapshot versions based on transaction timestamp. Key property: **Reads never block Writes, and Writes never block Reads.**

4. Case Study: Oracle Database Concurrency Control & Architecture

Oracle RDBMS relies on an advanced implementation of MVCC and UNDO segments to achieve high throughput without lock escalation:

- **System Change Number (SCN):** A logical timestamp incremented with every transaction commit. Used to ensure read consistency.
- **Multiversion Read Consistency:**
 - **Statement-Level Read Consistency:** Oracle guarantees that data returned by a single query is consistent with respect to the time the query started (SCN at query start).
 - **Transaction-Level Read Consistency:** Achieved via `SERIALIZABLE` isolation level. All statements see data consistent as of transaction start SCN.
- **Locking Architecture:** Uses Row-Level Locking (TX locks) for modified rows (stored in block headers, avoiding lock escalation memory limits) and Table-Level Locks (TM locks) to prevent structural schema changes during DML.
- **Core Architecture Processes:**
 - **DBWR (Database Writer):** Writes dirty buffers from memory (Buffer Cache) to disk files.
 - **LGWR (Log Writer):** Writes redo log entries from memory buffer to online Redo Log files on disk.
 - **CKPT (Checkpoint Process):** Signals DBWR at checkpoints and updates control file headers with SCN.
 - **Undo Tablespace:** Stores pre-image data used to construct old versions for consistent reads and transaction rollbacks.